A Reference Architecture for High-Availability Automatic Failover between PaaS Cloud Providers

Ivor D. Addo, Sheikh I. Ahamed
Marquette University
Milwaukee WI, USA
{ivor.addo, sheikh.ahamed}@marquette.edu

William C. Chu
Tunghai University
Taichung City, Taiwan
cchu@thu.edu.tw

Abstract – As the adoption rate of Cloud Computing continues to clamber on among various application archetypes, there is a growing concern for identifying reliable automatic failover solutions between various cloud providers in an attempt to minimize the effect of recent cloud provider outages among diverse always-on and mission-critical applications in healthcare, e-Commerce and ancillary settings. Automatic failover between cloud providers stands out as a solution for course-plotting application reliability requirements in support of high-availability, disaster recovery and high-performance scenarios. Using a case study involving Microsoft's Windows Azure cloud and the Google App Engine cloud solution, we investigate some of the key characteristics in this area of concern and present a reference architecture for automatic failover between multiple Platform-as-a-Service (PaaS) cloud delivery providers in a bid to maximize the delivery of architecturally significant quality attributes pertaining to High-Availability, Performance and Disaster Recovery in a mission-critical application prototype.

Keywords **– Cloud Reference Architecture; PaaS High Availability; Cloud Reliability; PaaS Disaster Recovery; Cloud-based Mission-Critical Applications; Automatic Failover;**

I. INTRODUCTION

Over the past three years, we have witnessed several cloud outages across multiple high-profile cloud providers who are otherwise considered to be widely reliable in terms of cloud service uptime. Unfortunately, when a cloud provider experiences an outage it can have a major financial and functional impact on mission-critical applications hosted on the provider's infrastructure. In 2013, some of the prominent cloud outages lasted anywhere from a five-minute failure to a week-long service disruption [1]. These unplanned outages can sometimes contribute unacceptably to an immeasurable cost to brand reputation and, resultantly, lost business. Reliability is quite indispensable in mission-critical systems and it can have a major toll on customer trust [2]. It goes without saying that, lost trust stemming from the lack of high-availability is not surprisingly a major deterrent to the adoption of cloud computing solutions of various kinds.

Some notable high-profile cloud service disruptions recorded towards the end of the year 2013 (that is, within a five-month span as illustrated in Figure 1) include [1]:

- *Facebook.com and Apps*: June 2013 – lasted thirty minutes affecting all Facebook hosted Apps and, of course, the social networking site as well.
- *Google.com and Google Cloud Apps:* The Cloud Apps outage occurred in July 2013 lasting forty minutes and affecting Gmail, Google Calendar, Google Talk, Google Drive, Google Documents, Google Spreadsheets and Google Presentations [3]. Also, in August, Google.com was down for about five minutes.
- *Microsoft's Outlook.com:* August 2013 – sporadic disruptions during a three-day period. In November 2013, sporadic issues lasting hours caused service disruptions for Windows Azure Cloud, Office365.com, other Microsoft web properties, and more dependent cloud consumer applications.
- *Amazon.com and AWS:* August 2013 – The initial outage affected Amazon.com, which happens to be an Ecommerce software-as-a-service platform – lasting about 45 minutes. A few days later, the Amazon Web Services (AWS) cloud reported performance degradation issues on its Elastic Computing (EC2) service as well as connectivity problems with its Elastic Load Balancing systems at the North Virginia datacenter. The issue emerged again in September lasting under two hours.
- *Apple's iCloud:* June and August 2013 – partial service disruption affecting a cross-section of its users. Some of the affected features include iMessage, Photo Stream, Documents in the Cloud, Backup and Restore, and iPhoto Journals.
- *Verizon's Terremark Cloud (Healthcare.gov):* in October 2013 the *Healthcare.gov* web site went down for several hours due to a service disruption in the Verizon cloud datacenter.

To highlight the severity of cloud outages on cloud consumer applications, we focused on examining the impact of cloud service disruptions on mission-critical systems like custom Ecommerce web sites and online healthcare applications that leverage cloud services.

While a number of enterprise organizations and individual cloud consumers often patronize solutions built on the *Software-as-a-Service* (SaaS) delivery model, most web-based cloud consumer applications are likely to leverage a *Platform-as-a-Service* (PaaS) cloud service delivery model. In the PaaS model, developers can leverage services that are built and maintained by the cloud provider. Consumers can easily upload their code to the cloud (for example, Google Cloud, Windows Azure, etc.) to take advantage of the automated high-scalability features inherent in the model, as the target application's usage intensifies [4] [5].

978-1-4799-6566-3/14 $31.00 © 2014 IEEE
DOI 10.1109/TSA.2014.12

IEEE computer society

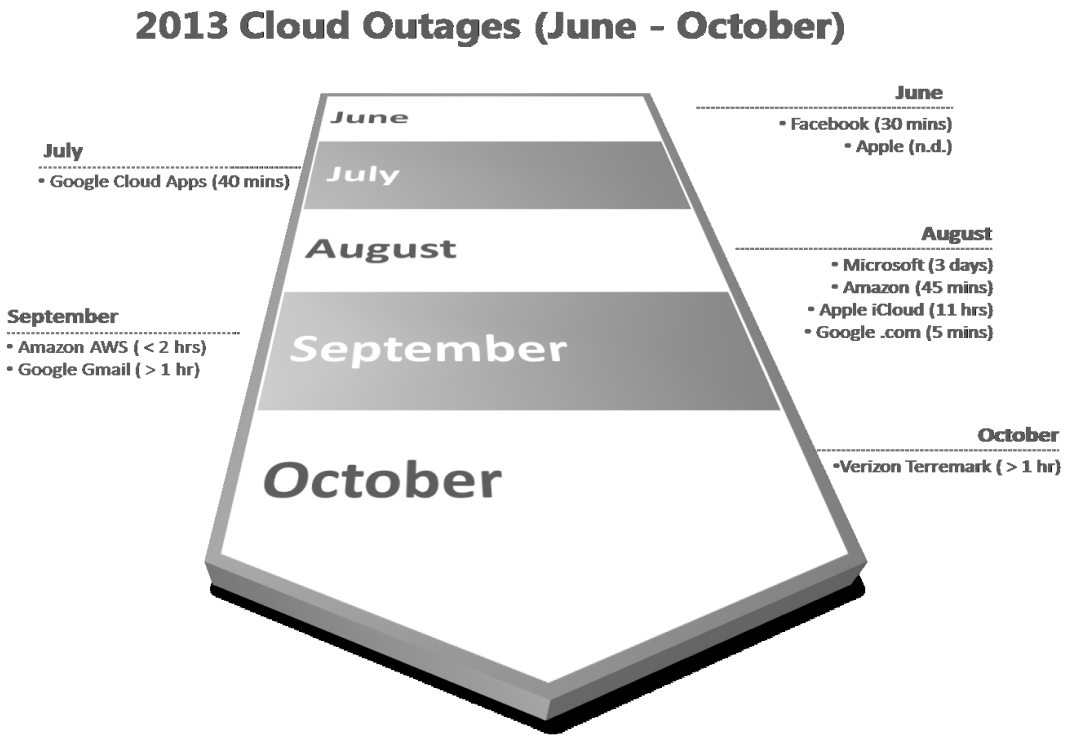

Fig. 1. High-Profile Public Cloud Outages between June and October 2013

For the sake of this study, we focused on the widely used *public cloud* deployment model.

Previous studies focused on a combination of datacenter redundancy solutions within a given cloud provider, server clustering, load-balancing, data replication, capacity planning strategies, aging infrastructure refresh policies within the cloud provider, and internal cloud monitoring and automation approaches for minimizing the impact of cloud service disruptions. However, it is important to note that some of the recent cloud outages transcend multiple datacenters hosted by a given cloud provider. In other words, some of the outages reported in the past couple of years took down an entire cloud provider (for example, Microsoft Azure).

As a result, we propose an automation strategy for monitoring external cloud services hosted on a given cloud provider's PaaS infrastructure, by utilizing ongoing data replication solutions between multiple cloud provider data stores while implementing an automatic failover solution to minimize the impact of cloud service disruptions on mission-critical applications. Knowing that public clouds often employ a *pay-as-you-go* model [4], we fashioned our proposed solution in a way that minimizes the cost of data replication between cloud providers. The proposed reference architecture can be employed to improve high-availability, fault-tolerance, disaster recovery, and resilience against performance degradation issues affecting a given cloud provider. To test the efficacy of this reference architecture, we evaluated a case study featuring a mission-critical movie recommendation and video streaming software solution prototype that is primarily hosted on the Windows Azure PaaS cloud solution. We simulated a number of cloud outages on the Windows Azure PaaS solution to test the proposed failover automation solution. The solution was involved with shifting incoming web site requests destined for the movie recommendations engine to a redundant Google App Engine PaaS solution that was capable of taking over the previous Azure PaaS workload to minimize downtime and ensure high-availability.

In the ensuing literature, we describe our main contributions by illustrating the proposed reference architecture for improving automatic failover in an inter-cloud-provider PaaS scenario while explaining our detailed solution design. We consider these design artifacts a key contribution since we are not aware of any existing study that provides a template architectural solution for this use case.

We describe our motivation for this study in section II and explain some of the main characteristics of the solution in section III. Section IV and V delve into the logical reference architecture and the implementation of our software solution prototype, respectively. A comparison of our approach with that of previous research work is presented in section VI. In section VII, we discuss our evaluation and findings in implementing the prototype application using our proposed reference architecture. In view of our findings, we share our conclusions in section VIII.

II. MOTIVATION

This study advances our recent investigations involving the application of public cloud services in electronic healthcare

(eHealth) scenarios. In addition, our ongoing research with scenarios entailing the use of the *Internet of Things* (IoT) depends largely on an "always-on" cloud storage infrastructure for persisting and analyzing streams of sensor related data.

In practice, most cloud vendors do not present easy opportunities for porting a given cloud hosted solution from one cloud vendor or provider to another. In this study, we explore some of the portability gaps in PaaS solutions today. The reference architecture goes beyond the high-availability problem in PaaS scenarios by also exploring interoperability and portability quality attributes for building software solutions that can be easily ported from one cloud provider to another. To afford our template solution the ability for automatic failover between PaaS vendors, we established a common platform for building the prototype solution which was, by itself, an interesting problem to explore.

Hassan and Holt [6] posit that establishing a reference architecture promotes re-use, reduces maintenance costs, and serves as a benchmark for software governance. We expect this reference architecture to serve cloud solution architects and developers in a similar fashion.

III. REQUIREMENTS AND CHARACTERISTICS

Some of the key non-functional requirements of our reference architecture include support for high-availability, high-scalability, high-performance, PaaS solution portability, and disaster recovery. However, in arriving at a reasonable solution we examine characteristics like Service Level Agreements, data or resource replication approaches, cost models, and dynamic domain name service (DNS) solutions. The key requirements at play in our proposed solution are not too far from some of the core tenets of cloud computing itself which Mell and Grance [10] described as having "massive scale, homogeneity, resilient computing, low cost software, virtualization, geographic distribution, service orientation, advanced security features [and] resource pooling."

A. Cloud Terminology

In the general, a "cloud" is a distinct information technology environment that is capable of remote rapid provisioning of scalable and measured IT resources [8] [9] [10]. Mvelase et al. posits that the most useful definition of cloud computing is that of the American National Institute of Standards and Technology (NIST) which states that, "Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction" [4].

For the sake of terminology, this literature often uses the terms *cloud provider* to represent the party that exposes cloud-based IT resources remotely (known as *cloud service*s) while the party that consumes these resources is referred to as the *cloud consumer*. Typically, cloud service usage conditions are expressed in a service contract between the provider and the consumer. This document is referred to as a *Service Level Agreement* (SLA) [8].

B. Cloud Business Model and Data Replication

The typical rationale for employing cloud computing is often the minimization or sometimes riddance of up-front hardware costs, software costs and operational costs [8]. Network bandwidth costs are also often cheaper. From a total cost of ownership (TCO) perspective, there are significant savings in both capital (Capex) and operational (Opex) expenditure. The attractive *pay-as-you-go* model is often employed. The significance of this model to our solution is that we seek to architect a technical solution that still keeps a *public cloud* solution attractive from a cost perspective even though we will be leveraging an *active-standy* data storage "cluster" across cloud providers through data replication to achieve automatic failover.

It is noteworthy that one of the hidden costs in cloud expenditure is bandwidth costs. These charges are often harder to estimate. In the case of Windows Azure, bandwidth costs apply when datacenter boundaries are crossed. As a result we focus our architecture on keeping the application and its associated data store in the same datacenter to minimize costs. In Figure 2, the green lines shown in the sequence diagram depict interactions across datacenters that will incur bandwidth costs. Interactions between the web site and the database will not incur data transfer costs [20].

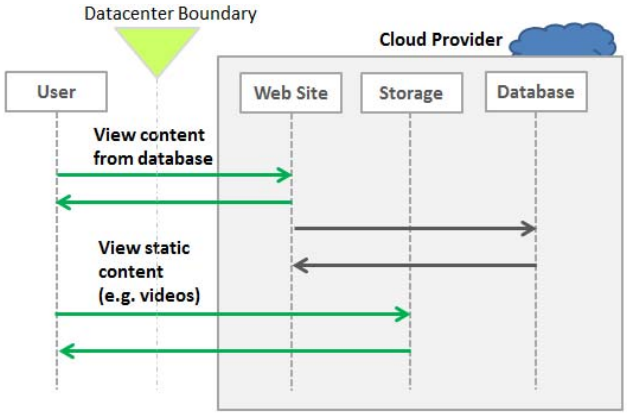

Fig. 2. Cloud Bandwidth Profile of our Video Streaming App

By introducing a background job for data replication across cloud providers, bandwidth costs will increase, though not considerably. A key tradeoff that must be balanced with improved availability is cost.

C. High Availability, Disaster Recovery and SLAs

Highly available (HA) applications are capable of withstanding instabilities in availability, performance and temporary failures [11]. To a large extent, the cloud consumer is expected to assess the SLAs of potential cloud providers prior to making a cloud hosting commitment. Though, most mission-critical applications are likely to have an SLA that cannot be met by the default SLAs offered by public cloud providers. Table 1 depicts a typical cloud service availability SLA offered by Microsoft's Windows Azure cloud (as of March 2014) [11]. While the target application can be

architected to run with limited capabilities when, for example, the SQL database component goes down, it is likely that the application will not be able to deliver its core functionality at all when the when the database is down for 43+ minutes in a 30-day month.

TABLE I. SAMPLE WINDOWS AZURE SLA

Azure Service	SLA	Potential Downtime Minutes/30 days
Compute	99.95%	21.6
SQL Database	99.90%	43.2
Storage	99.90%	43.2

A potential HA solution might be to consider a hybrid cloud deployment approach where the failover solution might leverage an on-premise solution (hosted by the cloud consumer). However, for some applications, it might be too expensive to explore an on-premise solution as a fallback plan, hence our proposed solution.

While HA deals with temporary degradation and failure conditions, disaster recovery (DR) is more concerned with rare catastrophic loss of solution functionality [11], potentially for a longer period of time. We fashioned our strategy to support both DR and HA scenarios. Some likely DR situations might include a network outage affecting the cloud provider, data corruption, datacenter down, cloud provider down, and more. We focused on the absolute worse-case scenario in the cloud provider continuum of disaster scenarios, which is the *cloud provider down* condition. Based on our proposed approach, the only unforeseeable DR condition that will remain unaddressed in terms of the availability of a mission-critical public-cloud PaaS hosted web resource, is a very exceptional condition where the entire internet might not be available.

IV. REFERENCE ARCHITECTURE

In creating our generalized *reference architecture* for improving high availability in *cloud-provider-down* conditions, we offer a cross-cloud hybrid PaaS solution illustrated in Figure 3 below.

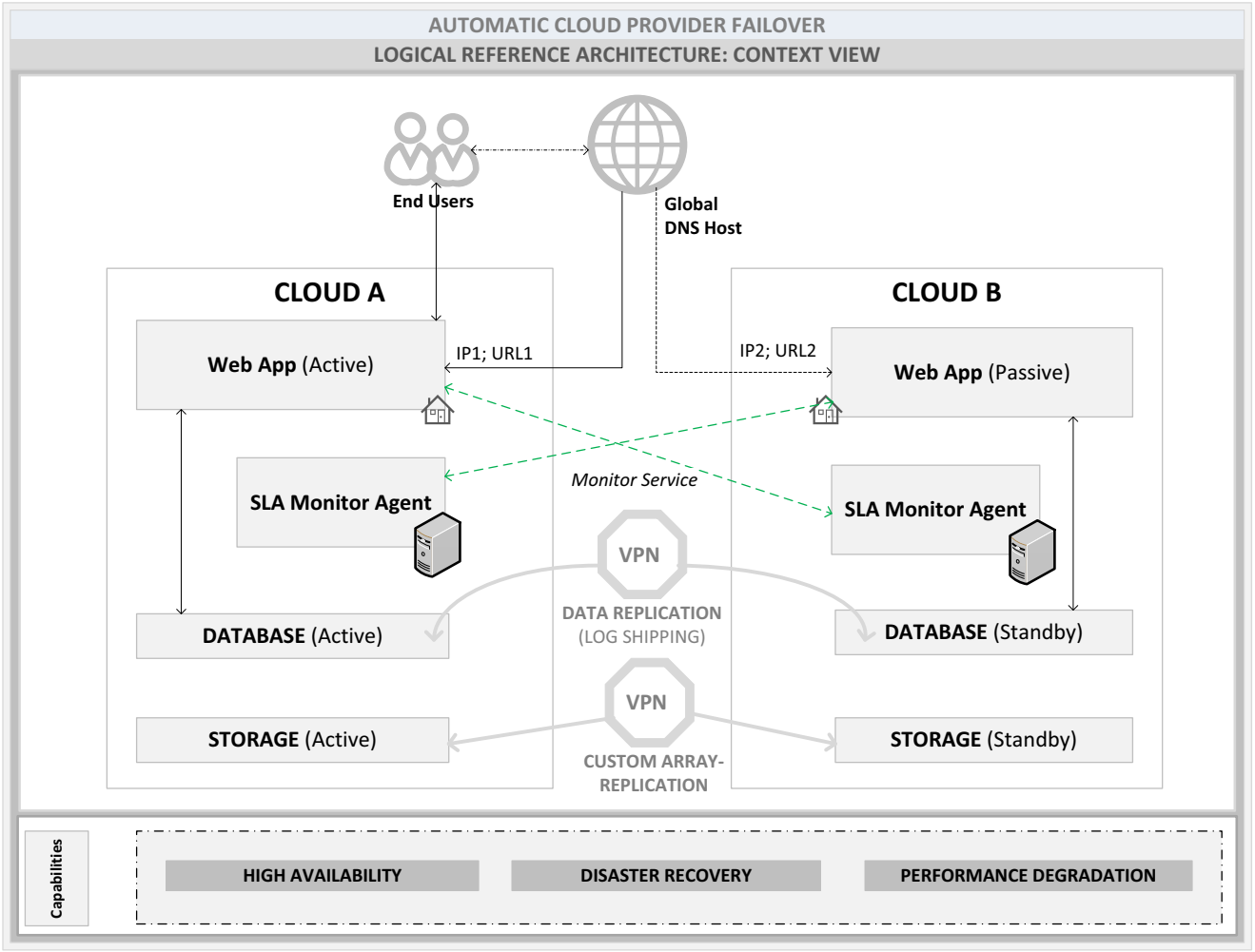

Fig. 3. Logical Reference Architecture for Cross Cloud Failover

Our model is predicated on the assumption that it is very rare for two major cloud providers to experience a down condition concurrently. As a result, the scope of our investigation is focused on utilizing two major public cloud PaaS providers. Though, in extreme cases, the model can be extended to accommodate more than 2 cloud providers.

A. Code Portability and Interoperability in PaaS

Due to wide variations in application programming interface (API) implementations across multiple cloud providers, we created an abstraction layer to port our application source code to meet the API requirements of both Cloud A and Cloud B. At the time of code deployment to the cloud, the build process generates two separate deployment packages that are, in turn, deployed to the various destination platforms.

B. SLA Monitor Agent

Our SLA Monitor Agent is a background process that is deployed on both cloud provider instances and remains accountable for monitoring performance and service disruptions on the cloud service hosted on an opposing cloud platform to maximize service uptime. For example, the SLA Monitor Agent on Cloud B will typically monitor the cloud service on Cloud A and vice versa. The monitoring agent is capable of failing over and failing back a cloud service automatically when a service disruption is discovered and subsequently when the service is restored, respectively.

The SLA Agent sends poll messages ($STATUS_1$ *to* $STATUS_k$) to the cloud service under investigation, for example, while monitoring the active *Web App* in Figure 3. The monitor then receives polling response messages ($STATUS\_TIME_1$ *to* $STATUS\_TIME_k$) that indicates the elapse time. When the cloud service is unreachable, the value "*TIMEOUT"* is returned. Based on the threshold parameters defined for retrying the poll process, the agent can determine whether or not the event was a very infinitesimal disruption (or a false positive) and, hence, does not warrant a failover.

In the event of a failover, the agent will notify the cloud consumer's administrator and also run automated scripts to complete the failover process (e.g. using Azure PowerShell scripting commands). All activities discovered by the SLA agent are logged into an SLA event log data store. The overall SLA reports and analysis of the cloud service can be reviewed through a data visualization interface.

C. Failover and Failback

We propose an *active-standby* data storage setup between the clouds. This is achieved through data replication between the two clouds. However, the web front-end service will be setup with an *active-passive* configuration and failover will only occur from Cloud A to Cloud B in an instance where the primary cloud, is no longer available or experiences service degradation. The architecture also supports automatic and manual failback scenarios where the system will redirect traffic from Cloud B back to Cloud A (the primary cloud) when A has been confirmed to be reliable and capable of taking on web traffic. During failback, data replication from cloud B to cloud A is initiated by the *SLA monitoring agent*. When the monitoring agent confirms that cloud A's data store is fairly consistent with that of cloud B, the front-end service failback process will be initiated.

D. Algorithm for Cloud Failover

We implemented a fault recovery algorithm that is synonymous with that of Saha and Mukhopadhyay's [13] proposed approach. The two *status* conditions under which a failover will occur include situations where performance degradation in a given Cloud Provider is detected by the SLA Monitoring Agent or the active Cloud Provider is down.

Inputs:

- n: number of cloud providers (for example, n=2)
- k: critical application dependent cloud services
- r: retry interval in milliseconds (for example, r=500)
- s: number of retries
- p: performance degradation threshold (milliseconds)

Logic for a given active cloud i, $\mathbf{0 <= i <= n - 1}$

if $CLOUD\_FAILOVER_i$ = FALSE; // *the active cloud*
 for all cloud services *k* executing in cloud *i*

  perform a $STATUS_k$ *check*;
  set $STATUS\_TIME_k$ to elapse time
  if $STATUS\_TIME_k > p$
   or $STATUS\_TIME_k$ == TIMEOUT
    set $FAULT\_FLAG_k$ = TRUE; *// save the fault on k*
    for *j* = 1 to *s*
      perform a $STATUS_k$ check;
      if $STATUS\_TIME_k > p$
       or $STATUS\_TIME_k$== TIMEOUT;
        if *j* == s *// retry complete*
          set $CLOUD\_FAILOVER_i$ = TRUE;
          initialize *FAILOVER*;
        else
          sleep for *r* milliseconds; // *retry to confirm*
          j = *j* + 1;
        end if
      else if a recent $FAULT\_FLAG_k = TRUE$ is found
        set $SPORADIC\_FLAG_k$ = TRUE;
        set $CLOUD\_FAILOVER_i$ = TRUE;
        initialize *FAILOVER*;
        exit for; *// note sporadic issues*
      else
        exit for; // *false alarm save fault on k and exit*
      end if
    end for
  end if

 end for

end if

Algorithm 1: Failover Fault Discovery for Cloud *i*

E. Recovery Time and Recovery Point Objectives

It is quite complex and near impossible to automatically switch between clouds with zero tolerance for data loss [11]. Hence, we defined a recovery time objective (RTO) and an associated recovery point objective (RPO) in both failover and failback scenarios. The cloud *Monitoring Agent* is focused on delivering the least amount of time for functionality recovery by obtaining a very low PTO.

F. Database and Storage Replication

Storage data replication is achieved through a custom array replication solution that is governed by the SLA Monitor Agent. Data replication is achieved through a custom log shipping process that is optimized to ensure that in the event of a service failover; only 15 minutes worth of data entered in Cloud *A* and pending replication to Cloud B (PTO) will be lost. To secure sensitive data transfers between the two clouds a virtual private network (VPN) is implemented.

G. Dynamic DNS (DDNS)

Domain names are typically hosted by multiple distributed data stores globally [14]. This makes it difficult to map a domain name that was previously pointing to Cloud A's IP address for a web application, to a new IP address presented by Cloud B's hosted web application during a failover. Using the Dynamic Domain Name Service (DDNS) feature, we were able to satisfy the requirement for implementing automated near-instantaneous DNS record updates to minimize downtime associated with domain name propagation globally. Because each PaaS cloud provider presents a unique IP address and domain name for the cloud-hosted PaaS web site, we employed a domain name that was agnostic to both cloud providers. This domain name was hosted by an external DNS service provider.

V. CASE STUDY: MOVIE RECOMMENDATIONS APP

We implemented a prototype solution for delivering web-based personalized movie recommendations and video streaming content. The web site was deployed to both the Google App Engine and the Windows Azure PaaS platform. An SLA Monitoring Agent was also deployed to both clouds, taking into account the reference architecture described earlier. The active cloud was defined as the Windows Azure cloud whereas Google's App Engine represented (Cloud B) the passive cloud platform.

The Movie Recommendation Application (App) features static image content which is stored on the cloud storage service and other Software-as-a-Service solutions as well as Facebook. Other data elements served on the web site are dynamic in nature and can be supplied to the end-user through a database call invoked by the web application based on the user's interaction with the application. Figure 2 depicts some of the cross-component interactions that are imminent in this scenario. We explore this example, as a mission-critical scenario, in the sense that if this type of web-based application were to take center-stage as a core entertainment resource for end-users, then the SLA expectations for the application could be comparable to that of Netflix, Google TV, Amazon Prime, and the likes. Figure 4 presents a screenshot of the movie recommendation application [19] used for the simulation test.

In evaluating the prototype solution, we selected the Google App Engine and Microsoft's Azure PaaS solutions because they currently offer support for multiple programming languages which makes it a bit easier to implement our deployment abstraction layer. Though, cloud vendor lock-in can be achieved by keeping PaaS APIs a bit difficult for porting code between clouds, we expect that with time most PaaS solution providers will improve the code portability quality attribute and further minimize the learning curve associated with pursuing this approach.

Fig. 4. Screenshot of the Movie Recommendations App [19]

VI. RELATED WORKS

While a few reference architectures exist for improving high-availability and disaster recovery within cloud implementations, with a focus on the Infrastructure-as-a-Service model, we are not aware of any literature that examines the challenges and characteristics involved with implementing a cross-cloud high-availability solution for PaaS scenarios. Both Erl et al. [8] and McKeown et al. [11] presented some interesting scenarios for implementing high-availability solutions within a given cloud provider. Additionally, McKeown et al. touched on the fact that exploring alternative clouds as a potential solution for improving HA and DR scenarios, in which the cloud provider is experiencing service disruption, seems feasible.

The authors posit that "the use of Virtual Machines [VMs] might be easier in these cases than cloud-specific PaaS designs" [11]. While the seemingly complex problem was attractive, as a research problem, we also realize that there are real-world scenarios where the use of an IaaS-based VM solution will not work well for mission-critical solutions that require a PaaS-based solution. There has been minimal discussion on how to implement a hybrid solution of this sort, some of the challenges and potential solutions in that space. Table II explores some of the differences and similarities between our work and that of other relevant studies.

TABLE II. COMPARISON WITH OTHER APPROACHES

CHARACTERISTICS	McKeown et. al. [11]	Erl et. al. [8]	Our proposed reference architecture
Presents Cloud availability characteristics	Yes	No	Yes
Presents Failover algorithm	No	Yes	Yes
Discusses HA and DR solutions	Yes	Yes	Yes
Supports PaaS scenarios	No	No	Yes
Discusses DDNS	No	No	Yes

VII. EVALUATION

To test the hypothesis that our reference architecture is feasible for most mission-critical PaaS hosted applications, we created a prototype of a web application as described in the case study. The executed simulations include:

- Cloud service outages, and
- Severe performance degradation introduced in the target application

Some of the key events logged in the SLA Monitor's log during the simulation test include:

- *Timeouts* – indicating that the cloud service was no longer accessible
- *Poor Elapse Time* – indicating how long it took to access a pre-defined critical transaction in the web application based on a pre-determined performance threshold
- *Cloud Failover* – due to a number of consecutive timeouts or poor elapse time notifications
- *Cloud Failback* – indicating that the application has been successfully failed back to the original Windows Azure PaaS platform

Our SLA Monitoring Agent was able to detect simulated server timeouts and software performance degradation failures with minute detection granularity. We were able to optimize the retry waiting period threshold value, the number of consecutive timeouts used for sifting out false positives, and the polling interval parameters to reach an optimal and performant solution that was capable of detecting service interruptions fairly quickly and, in succession, initiate the failover and notifications process.

To minimize the effect of network flooding and some degree of self-inflicted denial-of-service issues we adjusted the polling scheme so each subsequent retry will implement a multiplier of the original retry wait time till the maximum number of retries was reached.

For this experiment, we also relaxed the recovery time objective (RTO) to minimize the potential cost of replicating multiple videos and associated data across the VPN connection. In our experiments, the longest period of time tracked for performing a full cutover from Cloud A to Cloud B was recorded in the execution time for DNS update replication in some geographically dispersed user test cases.

VIII. CONCLUSIONS

Based on the findings from our HA and DR simulation tests, we described a template solution architecture for implementing automatic failover between PaaS cloud provider platforms. In our study, we highlighted opportunities for using DDNS, an abstraction layer for cross-cloud deployment solutions and a custom replication solution for achieving high availability failover for mission-critical solutions that seek higher SLAs in cloud systems. Based on some of our lessons learned, we presented an algorithm to help shape future implementations of automatic failover in PaaS scenarios. Needless to say, with growing concerns regarding recent cloud service disruptions among high-profile cloud providers, it is in the interest of cloud consumers who employ the PaaS model to consider adopting and extending the reference architecture presented in this literature.

ACKNOWLEDGEMENTS

This work is partly sponsored by TUNGHAI UNIVERSITY "The U-Care ICT Integration Platform for the Elderly – No.102 GREEnS004 – 2," 2013.

REFERENCES

[1] J. R Raphael, "The Worst Cloud Outages of 2013," *InfoWorld*, December 2013. Retrieved from http://www.infoworld.com/slideshow/133109/the-worst-cloud - outages-of-2013-part-2-232912.

[2] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, W. Vogels, "Dynamo: Amazon's Highly Available Key-value Store," *SOSP*, pp. 205-220, 2007.

[3] Google, "Apps Status Dashboard," *Google Apps*, n. d., Retrieved from http://www.google.com/appsstatus#hl=en&v=status.

[4] P. Mvelase, N. Dlodlo, Q. Williams, M. Adigun, "Custom-made Cloud Enterprise Architecture for Small and Micro Enterprises," 589-601, *Grid and Cloud Computing*, vol. 2, 2012.

[5] C. Weinhardt, A. Anandasivam, B. Blau, J. Stober, "Business Models in the Service World," IEEE Computer, vol. 11, no. 2, pp. 28-33, 2009.

[6] A. E., Hassan, R. C. Holt, "A Reference Architecture for Web Servers," *Reverse Engineering, 2000. Proceedings. Seventh Working Conference*, pp.150-159, 2000.

[7] Facebook, “Using Graph API,” *Facebook*, n.d.

[8] T. Erl, Z. Mahmood, R. Puttini, “Cloud Computing Concepts, Technology, Architecture,” *Prentice Hall*, pp. 110-160, 2013.

[9] P. Mell, T. Grance, “A NIST definition of Cloud Computing,” *National Institute of Standards and Technology, NIST*, 2009.

[10] P. Mell, T. Grance, “Effectively using the Cloud Computing Paradigm,” *Gaithersburg, MD: NIST Information Technology, Laboratory,* 2009.

[11] M. McKeown, H. Kommalapati, J. Roth, “Disaster Recovery and High Availability for Azure Applications”, *Microsoft Azure – MSDN,* April 2014. Retrieved from http://msdn.microsoft.com/library/azure/dn251004.aspx.

[12] I. Saha, D. Mukhopadhyay, S. Banerjee, “Designing Reliable Architecture for State-ful Fault Tolerance,” In Proceedings of *Seventh International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT'06)*, pp. 545-551, 2006.

[13] I. Saha, D. Mukhopadhyay, “A Distributed Algorithm of Fault Recovery for Stateful Failover,” *Theory and Applications of Models of Computation – TAMC, Lecture Notes in Computer Science: Springer*, vol. 4484, pp. 738-749, 2007.

[14] C. Chi-Chung, Y. Man-Ching, A. Yip, "Dynamic DNS for load balancing," *Distributed Computing Systems Workshops, 2003. Proceedings. 23rd International Conference,* pp. 962, 965, 19-22, May 2003.

[15] W. Itani, A. Kayssim, A. Chehab, “Privacy as a Service: Privacy-Aware Data Storage and Processing in Cloud Computing Architectures,” *DASC '09 Proceedings of the 2009 Eighth IEEE International Conference on Dependable, Autonomic and Secure Computing*, pp. 711-716, 2009.

[16] G. Zhao, Z. Li, W. Li, H. Zhang, Y. Tang, "Privacy Enhancing Framework on PaaS," *Cloud and Service Computing (CSC), 2012 International Conference on*, pp.131-137, 22-24, Nov. 2012.

[17] I. D. Addo, S. I. Ahamed, W. C. Chu, “Toward Collective Intelligence for Fighting Obesity,” *COMPSAC*, 690-695, 2013.

[18] National Institute of Standards and Technology, “NIST Cloud Computing Standards Roadmap,” *NIST - US Department of Commerce,* 2013.

[19] I. Addo, S. Ahamed, S. Yau, B. Buduru, “A Reference Software Architecture for Improving Security and Privacy in Cloud-Enabled IoT Applications,” *Submitted to MS 2014*, 2014.

[20] D. Pallmann, “Hidden Costs in the Cloud, Part 2: Windows Azure Bandwidth Charges,” 2010. Retrieved from http://davidpallmann.blogspot.com/2010/08/hidden-costs-in-cloud-part-2-windows.html.